

Query Performance

Instructor: Aditya Parameswaran

Covers Lecture(s): 7 – 8

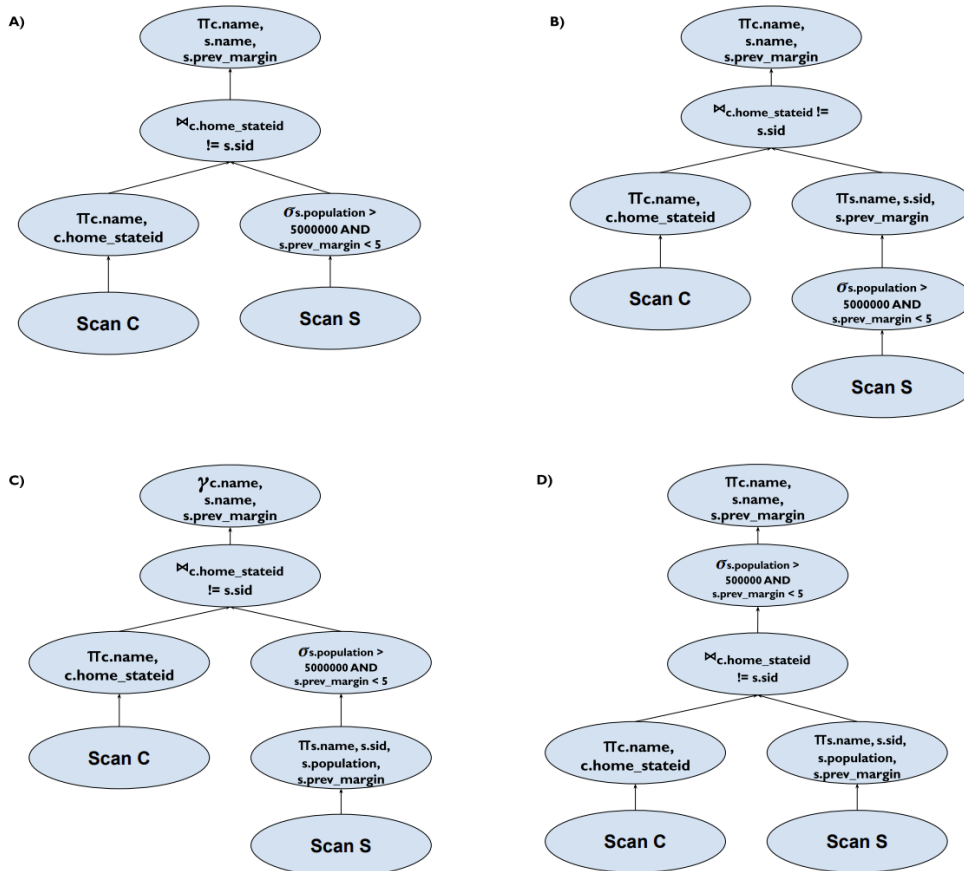
I Query Optimization

1. Consider the following SQL query about electoral candidates. Which of the query plans is the optimized query plan for this query? What optimizations were used?

```

SELECT c.name, s.name, s.prev_margin
FROM candidates AS c INNER JOIN states AS s
ON c.home_stateid != s.sid
WHERE s.population > 5000000 AND s.prev_margin < 5;

```



II Join orders [Optional]

2. Use the following schema and information for this question.

- `students` (`student_id`, `name`, `age`, `major`) with 100 tuples
- `orders` (`student_id`, `book_id`, `date`) with 500 tuples
- `books` (`book_id`, `author`, `num_copies`) with 200 tuples

We want to do a natural join of the above three relations. What is the optimal join order?

III Query Plan Rewriting, Part I

Consider the following schema:

- `teams` (`id`, `state`)
- `results` (`team_id`, `opponent_id`, `win`)

And the following SQL query retrieves a list of team IDs along with the IDs of opponents they defeated:

```
SELECT t.id, r.opponent_id
FROM teams AS t INNER JOIN results AS r
ON t.id = r.team_id
WHERE r.win = 1
```

3. *Logical Plan.* Write out the logical plan based on order of evaluation, prior to optimization. Write it as a tree of relational algebra operators. (Hint: The order of the operators should be join, filter, then projection.)

4. *Predicate Push down.* One way to optimize this query plan is to perform predicate push down, where we move the σ around. What would the logical plan look like if we push the predicate through the join?

5. Why may we want to use the second query plan instead of the first?

6. **Challenge:** Note that we have not placed any foreign key constraints. What is a situation in which the first query plan would perform better? *Hint:* Consider a scenario where the **results** table has invalid **team_id** values that don't match entries in the **teams** table.

IV Query Plan Rewriting, Part II

Next, let's consider a somewhat more complicated query.

```
SELECT t.id, r.opponent_id
FROM teams AS t INNER JOIN results AS r
ON t.id = r.team_id
WHERE t.state = 'California' AND r.win = 1
```

Let's walk through the query optimization process again.

7. What does the logical query plan look like if we compute the join first, then filter by location, and then filter by wins?

8. Assume that after computing the joins, there are 50 rows that are in California, 100 rows that are wins, and 10 rows that are both. Without pushing predicates through the join, what is the optimal logical plan? (there are no indices available)

9. Now assume there are 1000 teams total (10% of which are from California) and 5000 results total (half of which are wins). We have a hash index on both `teams` and `results` for the `id` and `team_id` columns, respectively. What is the optimal logical plan?

V EXPLAIN ANALYZE with Joins

Consider the following schema:

- `actors (id, name)`
- `cast_info (person_id, movie_id)`

10. Write a query that computes an inner join of `actors` and `cast_info` on the actor ID. Your query should return all attributes. What kind of join is performed? Why does the query optimizer choose this join method?

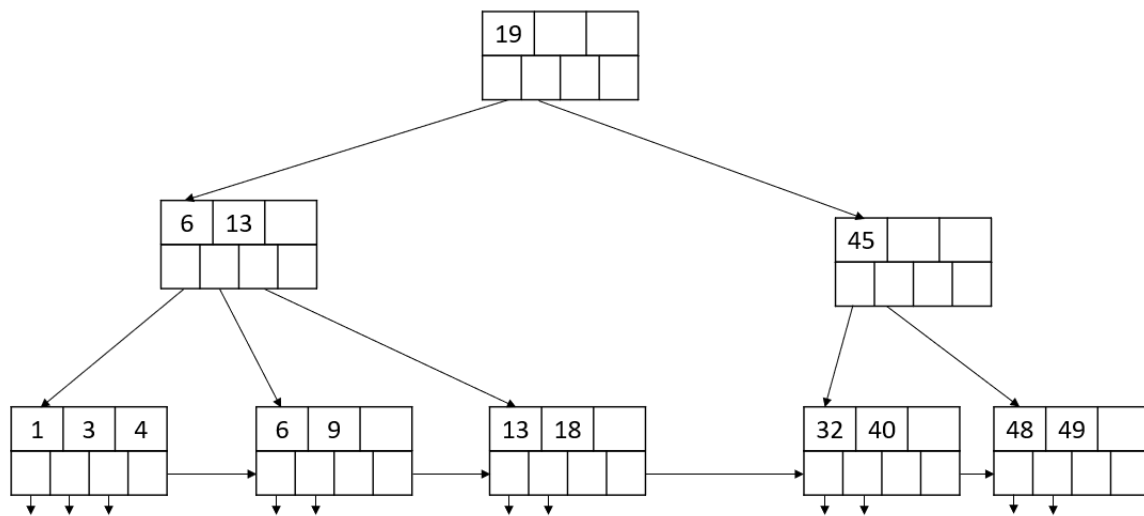
11. Which settings are related to selecting the type of join that the query optimizer can select?

12. Let's turn off hash join as an option for the query optimizer. Rerun the same query in Question 10. Now what is the join the query optimizer picks? Why might this be preferred over the remaining alternatives?

13. Write a SQL command that turns off the join method used in the previous question. What do you think the query optimizer will use now?

14. Write two SQL commands that reenables the two join methods you disabled in previous questions.

VI Indexes [Optional]



15. Describe how to navigate a B+ tree in general.

16. Describe the traversal route if we search for 48 in the given B+ tree above. How many nodes are reached?

17. Describe how to do a range search on keys 5 through 20 inclusive in the given B+ tree above. How many nodes are reached?